Lab 3: dane z TMDB

Zaawansowane programowanie na platformę iOS

Na tych zajęciach aplikacja zaczyna pobierać prawdziwe filmy z TMDB. Lista i wyszukiwanie dostają dane z sieci. Ekran szczegółów pokazuje nowe informacje: hasło filmu, czas trwania, kategorię wiekową, reżysera, zwiastun, obsadę, serwisy z filmem i podobne filmy.

Przykładowe filmy z `Services/SampleMovies.swift` znikają. Testy dostają własne filmy w `TestMovies.swift`, a podglądy w Xcode pytają TMDB.

Cały kod piszą Państwo sami. Ta instrukcja mówi, co ma powstać i gdzie. Nazwy plików, typów i funkcji proszę zachować, bo według nich sprawdzam prace.

Zanim Państwo zaczną

1. Lab 2 musi być skończony.
2. Klucz TMDB musi być w `Config/Secrets.xcconfig`. Instrukcja jest w README, w sekcji Klucze API. W kodzie klucz zwraca funkcja `tmdbAPIKey()`.

Pliki

Nowe:

`MyNextMovie/Services/TMDB.swift`	zapytania do TMDB
`MyNextMovie/Models/MovieDetails.swift`	szczegóły filmu
`MyNextMovie/ViewModels/MovieDetailViewModel.swift`	stan ekranu szczegółów
`MyNextMovie/Views/MovieDetailSections.swift`	sekcje pod opisem filmu
`MyNextMovie/Views/Components/PopularMoviesPreview.swift`	filmy z TMDB w podglądach
`MyNextMovieTests/TestMovies.swift`	filmy do testów
`MyNextMovieTests/TMDBTests.swift`	testy zapytań
`MyNextMovieTests/MovieDetailsTests.swift`	testy szczegółów
`MyNextMovieTests/MovieDetailViewModelTests.swift`	testy view modelu

Do zmiany:

`MyNextMovie/Services/MovieLoader.swift`	nowy typ `MovieDetailsLoader`
`MyNextMovie/Views/MovieDetailView.swift`	ładowanie i pokazywanie szczegółów
`MyNextMovie/Views/MovieListView.swift`	parametr `loadDetails`, szare karty w czasie ładowania
`MyNextMovie/Views/SearchView.swift`	parametr `loadDetails`
`MyNextMovie/MyNextMovieApp.swift`	funkcje TMDB zamiast przykładowych podglądy i testy

Do usunięcia: `MyNextMovie/Services/SampleMovies.swift`.

Nowe pliki w folderach `MyNextMovie/` i `MyNextMovieTests/` same trafiają do projektu. Wystarczy je utworzyć w Xcode (File, New, File from Template, Swift File) we właściwym folderze.

Jak działa TMDB

TMDB to REST API. Aplikacja wysyła zapytanie `GET` pod adres i dostaje odpowiedź w JSON. Lista popularnych filmów:

```
https://api.themoviedb.org/3/movie/popular?api_key=KLUCZ
```

Odpowiedź (skrócona):

```
{
  "page": 1,
  "results": [
    {
      "id": 603,
      "title": "The Matrix",
      "overview": "A hacker learns that the world is a simulation.",
      "release_date": "1999-03-31",
      "poster_path": "/f89U3ADr1oiB1s9GkdPOEpXUk5H.jpg",
      "vote_average": 8.2,
      "genre_ids": [28, 878]
    }
  ]
}
```

Każde zapytanie to trzy kroki:

1. Adres. Składa go `URLComponents`: `scheme`, `host`, `path` i `queryItems`. `URLComponents` zamienia spację i polskie litery na znaki dozwolone w adresie, np. `pan tadeusz` na `pan%20tadeusz`. Proszę nie sklejać adresu ze stringów.
2. Wysłanie. `try await URLSession.shared.data(from: adres)` zwraca parę `(data, response)`. Funkcja jest `async`, więc ekran w tym czasie działa. `response` rzutowany na `HTTPURLResponse` ma `statusCode`: `200` znaczy OK, `401` zły klucz. `URLSession` nie rzuca błędu przy odpowiedzi 404, kod trzeba sprawdzić samemu.
3. Dekodowanie. `JSONDecoder` zamienia JSON na struktury Swift, które są `Decodable`. Klucze w JSON są w snake_case (`release_date`), a pola w Swift w camelCase (`releaseDate`). Zamienia je `decoder.keyDecodingStrategy = .convertFromSnakeCase`. `Movie` z lab 1 jest już `Codable`.

Pola w JSON, których struktura nie ma, dekoder pomija. Struktura może więc mieć tylko te pola, których aplikacja potrzebuje.

Funkcje z TMDB mają mieć te same typy co funkcje z przykładowymi danymi: `MovieLoader` i `MovieSearch` z `Services/MovieLoader.swift`. Wtedy view modele i widoki się nie zmieniają. Zmienia się tylko funkcja podana w `MyNextMovieApp.swift`.

Dokumentacja zapytań: https://developer.themoviedb.org/reference/intro/getting-started

Zadanie 1: zapytania do TMDB

`Services/TMDB.swift`

Stałe zamiast wpisanych w kod napisów: schemat `https`, host `api.themoviedb.org`, ścieżki `/3/movie/popular`, `/3/search/movie`, `/3/discover/movie` i nazwy parametrów `api_key`, `query`, `with_genres`. Kody HTTP też jako stałe: `httpStatusOK = 200`, `httpStatusUnauthorized = 401`.

Typy:

- `TMDBMoviePage`: struktura `Decodable` z jednym polem `results: [Movie]`. To jedna strona listy z TMDB.
- `TMDBError`: struktura z polem `message: String`, zgodna z `LocalizedError`. Jej `errorDescription` zwraca `message`. Dzięki temu `error.localizedDescription` w view modelu pokaże ten tekst na ekranie.

Typy, które dekoduje `JSONDecoder`, proszę oznaczyć `nonisolated`, tak jak `Movie`. Aplikacja domyślnie działa na głównym wątku (`MainActor`), a `nonisolated` pozwala dekodować je także poza nim.
Funkcje:

Funkcja	**Co robi**
`tmdbURL(path: String, parameters: [URLQueryItem], apiKey: String) -> URL?`	adres z kluczem `api_key` jako pierwszym parametrem, potem `parameters`
`tmdbPopularURL(apiKey: String) -> URL?`	adres listy popularnych
`tmdbSearchURL(query: String, genreId: Int, apiKey: String) -> URL?`	z tekstem: `/search/movie` z `query`. Bez tekstu: `/discover/movie` z `with_genres`
`decodeTMDBMovies(_ data: Data) throws -> [Movie]`	`results` z `TMDBMoviePage`
`checkTMDBResponse(_ response: URLResponse) throws`	dla `401` rzuca `TMDBError` z tekstem o kluczu API, dla innego kodu niż `200` z numerem kodu
`fetchPopularMovies() async throws -> [Movie]`	ma typ `MovieLoader`, pobiera popularne filmy
`searchTMDBMovies(query: String, genreId: Int) async throws -> [Movie]`	ma typ `MovieSearch`, szuka filmów

Zasady:

- Gdy `tmdbAPIKey()` jest puste, funkcje `fetch...` i `search...` rzucają `TMDBError` z komunikatem „Missing TMDB API key. Add it to Config/Secrets.xcconfig.”.
- `/search/movie` nie umie filtrować po gatunku. `searchTMDBMovies` filtruje wynik po gatunku Państwa funkcją `filterMovies` z lab 2, z pustym tekstem.
- Przed zapytaniem proszę obciąć spacje wokół tekstu, jak w lab 2.
- Wysłanie, sprawdzenie kodu i zwrot danych proszę wydzielić do jednej prywatnej funkcji, np. `fetchTMDBData(_ address: URL?) async throws -> Data`. Wszystkie zapytania z niej korzystają.

`MyNextMovieApp.swift`

W `RootView` proszę podać `fetchPopularMovies` zamiast `loadSampleMovies` i `searchTMDBMovies` zamiast `searchSampleMovies`.

Testy w `MyNextMovieTests/TMDBTests.swift`

Struktura `TMDBTests`, oznaczona `@MainActor`, jak pozostałe testy. Żaden test nie łączy się z siecią. JSON do testów proszę wpisać w pliku testów jako napis wielowierszowy (`"""`), np. skróconą odpowiedź z początku tej instrukcji.

Test	Sprawdza
`decodesSnakeCaseKeys`	`releaseDate`, `posterPath`, `voteAverage`, `genreIds` po dekodowaniu
`decodesMissingPosterAsNil`	`"poster_path": null` daje `nil`
`decodingFailsForOtherJSON`	inny JSON, np. `{"status_code": 7}`, rzuca `DecodingError`
`popularURLHasAPIKey`	adres to dokładnie `https://api.themoviedb.org/3/movie/popular?api_key=test-key`
`searchWithTextUsesSearchEndpoint`	z tekstem ścieżka `/3/search/movie` i parametr `query`
`searchWithGenreOnlyUsesDiscoverEndpoint`	bez tekstu ścieżka `/3/discover/movie` i parametr `with_genres`
`searchTextIsPercentEncoded`	„pan tadeusz” daje w adresie `query=pan%20tadeusz`
`detailsURLAppendsEveryPart`	adres szczegółów: ścieżka `/3/movie/603` i wszystkie części w `append_to_response` (zadanie 2)
`okResponsePasses`	kod `200` nie rzuca błędu
`unauthorizedResponseMentionsAPIKey`	kod `401` rzuca `TMDBError` z „API key” w treści
`otherErrorResponseShowsStatus`	kod `404` rzuca `TMDBError` z „404” w treści

Podpowiedzi:

- Odpowiedź z dowolnym kodem: `HTTPURLResponse(url: adres, statusCode: 401, httpVersion: nil, headerFields: nil)`.
- Błąd rzucony w teście: `let thrownError = #expect(throws: TMDBError.self) { try checkTMDBResponse(response) }`. `thrownError` to złapany błąd albo `nil`.
- Opcjonalny adres rozpakowuje `let searchURL = try #require(tmdbSearchURL(...))`. Gdy jest `nil`, test kończy się błędem zamiast awarii.
- Ścieżkę adresu zwraca `searchURL.path()`, a parametry `URLComponents(url: searchURL, resolvingAgainstBaseURL: false)?.queryItems`.

Proszę sprawdzić w aplikacji:

1. Lista pokazuje popularne filmy z plakatami.
2. Wyszukiwanie „matrix” znajduje filmy z TMDB, nie tylko pięć przykładowych.
3. Sam gatunek Animation, bez tekstu, pokazuje filmy animowane.
4. Bez internetu (wyłączone Wi-Fi w Macu) lista pokazuje błąd i przycisk „Try again”.

Gdy zamiast filmów jest błąd „Missing TMDB API key”, klucza nie ma w `Config/Secrets.xcconfig`. Po dodaniu klucza proszę zrobić Product, Clean Build Folder (⇧⌘K).

Zadanie 2: szczegóły filmu

Szczegóły przychodzą w jednym zapytaniu. Parametr `append_to_response` dokleja do odpowiedzi obsadę, filmy wideo, kategorie wiekowe, serwisy streamingowe i polecane filmy:

```
/3/movie/603?
api_key=KLUCZ&append_to_response=credits,videos,release_dates,watch/
providers,recommendations
```

Odpowiedź (skrócona):


```
{
  "id": 603,
  "tagline": "Believe the unbelievable.",
  "runtime": 136,
  "credits": {
    "cast": [
      {"id": 6384, "name": "Keanu Reeves", "character": "Neo", "profile_path":
"/8RZL.jpg"}
    ],
    "crew": [
      {"id": 9340, "name": "Lana Wachowski", "job": "Director"}
    ]
  },
  "videos": {
    "results": [
      {"key": "vKQi3bBA1y8", "site": "YouTube", "type": "Trailer", "official":
true}
    ]
  },
  "release_dates": {
    "results": [
      {"iso_3166_1": "PL", "release_dates": [{"certification": "16"}]}
    ]
  },
  "watch/providers": {
    "results": {
      "PL": {
        "link": "https://www.themoviedb.org/movie/603-the-matrix/watch?locale=
PL",
        "flatrate": [
          {"provider_id": 119, "provider_name": "Amazon Prime Video",
"logo_path": "/gMZd.png"}
        ]
      }
    }
  },
  "recommendations": {
    "results": [ ...filmy jak w liście popularnych... ]
  }
}
```


Pełną odpowiedź zobaczą Państwo, wklejając adres z kluczem do przeglądarki.

`Models/MovieDetails.swift`

Struktury `Decodable`, oznaczone `nonisolated`, odpowiadające JSON powyżej:

Struktura	**Pola**
`MovieDetails`	`id`, `tagline`, `runtime: Int?`, `credits`, `videos`, `releaseDates`, `watchProviders`, `recommendations: TMDBMoviePage`
`Credits`	`cast: [CastMember]`, `crew: [CrewMember]`
`CastMember`	`id`, `name`, `character`, `profilePath: String?`. Zgodna z `Identifiable` dla `ForEach`
`CrewMember`	`id`, `name`, `job`
`VideoList`	`results: [Video]`
`Video`	`key`, `site`, `type`, `official: Bool`
`ReleaseDateList`	`results: [CountryReleaseDates]`
`CountryReleaseDates`	`countryCode`, `releaseDates: [ReleaseDate]`
`ReleaseDate`	`certification`
`WatchProviderList`	`results: [String: CountryWatchProviders]`, klucz to kod kraju, np. „PL”
`CountryWatchProviders`	`link`, `flatrate`, `rent`, `buy`, trzy ostatnie typu `[WatchProvider]?`
`WatchProvider`	`providerId`, `providerName`, `logoPath: String?`. `Identifiable` przez `var id: Int { return providerId }`

`runtime` jest opcjonalne, bo TMDB nie zna długości filmów zapowiedzianych. Listy serwisów są opcjonalne, bo TMDB pomija te, których kraj nie ma.

Klucz `watch/providers` ma ukośnik, więc nie może być nazwą pola w Swift. Proszę dodać w `MovieDetails` enum `CodingKeys: String, CodingKey` z `case watchProviders = "watch/providers"`. Gdy struktura ma `CodingKeys`, trzeba w nich wymienić wszystkie pola, także te bez zmiany nazwy. `CodingKeys` mówi dekoderowi, które pole odpowiada któremu kluczowi.

Tak samo w `CountryReleaseDates`: klucz `iso_3166_1` po `.convertFromSnakeCase` to `iso31661`. Pole nazywa się `countryCode`, więc `case countryCode = "iso31661"`. Stałe:

- `emptyMovieDetails`: szczegóły z pustymi listami i `runtime: nil`. Ekran pokazuje je, zanim przyjdą prawdziwe.
- `noWatchProviders`: `CountryWatchProviders` bez serwisów, dla kraju bez danych.
- `watchRegionCode`: kraj z ustawień telefonu, `Locale.current.region?.identifier`, a gdy go nie ma, `"PL"`.

Funkcje:

Funkcja	**Wynik**
`formattedRuntime(_ details: MovieDetails) -> String`	136 → „2h 16m”, 45 → „45m”, nieznany albo 0 → `""`
`directorNames(_ details: MovieDetails) -> String`	osoby z `job` równym „Director”, każda raz, rozdzielone `", "`
`ageCertification(_ details: MovieDetails, regionCode: String) -> String`	pierwsza niepusta `certification` w danym kraju, `""` gdy brak
`trailerKey(_ details: MovieDetails) -> String`	`key` filmu z YouTube typu „Trailer”. Oficjalny ma pierwszeństwo, gdy go nie ma, pierwszy nieoficjalny. `""` gdy brak
`trailerURL(_ details: MovieDetails) -> URL?`	`https://www.youtube.com/watch?v=` plus klucz, `nil` gdy brak
`hasTrailer(_ details: MovieDetails) -> Bool`	czy jest zwiastun
`movieFactsLine(_ movie: Movie, _ details: MovieDetails, regionCode: String) -> String`	„1999 · ★ 8.2 · 2h 16m · 16”, bez części, których nie ma
`watchProviders(_ details: MovieDetails, regionCode: String) -> CountryWatchProviders`	serwisy w kraju, `noWatchProviders` gdy brak
`hasWatchProviders(_ countryProviders: CountryWatchProviders) -> Bool`	czy jest choć jeden serwis
`profileURL(_ castMember: CastMember) -> URL?`	zdjęcie aktora: `https://image.tmdb.org/t/p/w185` plus `profilePath`, `nil` gdy brak
`logoURL(_ provider: WatchProvider) -> URL?`	logo serwisu: `https://image.tmdb.org/t/p/w92` plus `logoPath`, `nil` gdy brak

`movieFactsLine` zaczyna od `yearAndRating` z lab 1. Przed załadowaniem szczegółów daje więc samo „1999 · ★ 8.2”.

TMDB często wymienia jedną osobę kilka razy, np. jako reżysera i scenarzystę. Stąd „każda raz” w `directorNames`.

`Services/TMDB.swift`

Dopisać:

- `tmdbMovieDetailsURL(movieId: Int, apiKey: String) -> URL?`: ścieżka `/3/movie/` plus id, parametr `append_to_response` z wartością `credits,videos,release_dates,watch/providers,recommendations`.
- `decodeTMDBMovieDetails(_ data: Data) throws -> MovieDetails`.
- `fetchMovieDetails(movieId: Int) async throws -> MovieDetails`.

`Services/MovieLoader.swift`

Nowy typ funkcji:

```
typealias MovieDetailsLoader = (_ movieId: Int) async throws -> MovieDetails
```

`ViewModels/MovieDetailViewModel.swift`

Klasa `MovieDetailViewModel`, `@Observable`, wzorowana na `MovieListViewModel`:

- pola `state: LoadState`, `details: MovieDetails` (na start `emptyMovieDetails`), `errorMessage: String`, wszystkie `private(set)`,
- `init(movieId: Int, loadDetails: @escaping MovieDetailsLoader)`,
- `loadIfNeeded()` i `load()`, oba `async`, jak w `MovieListViewModel`. `load` woła `loadDetails(movieId)`.

Ekran szczegółów

`MovieDetailView`:

- `init(movie: Movie, loadDetails: @escaping MovieDetailsLoader)`. W `init` tworzy view model: `_viewModel = State(initialValue: MovieDetailViewModel(...))`. Tak samo `MovieListView` dostaje swój.
- `.task { await viewModel.loadIfNeeded() }` na `ScrollView`.
- `MovieInfo` dostaje też `details`. Pod tytułem hasło filmu (`tagline`) kursywą, potem `movieFactsLine` zamiast `yearAndRating`. Pod opisem „Directed by" i reżyserzy. Każda z tych części tylko wtedy, gdy ma dane. Do załadowania szczegółów są puste, więc ekran wygląda jak w lab 2.
- Pod `MovieInfo`, zależnie od `viewModel.state`: `ProgressView` w trakcie ładowania, `MovieDetailSections` po załadowaniu, komunikat z przyciskiem „Try again" po błędzie. Film z listy zostaje na ekranie także przy błędzie.

`MovieListView` i `SearchView` dostają parametr `loadDetails: MovieDetailsLoader` w `init` i przekazują go do `MovieDetailView` w `.navigationDestination`. `MyNextMovieApp.swift` podaje `fetchMovieDetails`.
`Views/MovieDetailSections.swift`, widok `MovieDetailSections(details: MovieDetails)`. Każda sekcja tylko wtedy, gdy ma dane:

1. Przycisk „Watch Trailer". To `Link(destination:)` z adresem zwiastuna, otwiera YouTube.
2. „Cast": najwyżej 15 aktorów w wierszu przewijanym w bok. Okrągłe zdjęcie (`AsyncImage`, `.clipShape(Circle())`), pod nim imię i nazwisko oraz postać. Bez zdjęcia ikona `person.fill`.
3. „Where to Watch": serwisy w kraju `watchRegionCode`, w grupach „Stream" (`flatrate`), „Rent" i „Buy". Każdy serwis to logo i nazwa, grupa to wiersz przewijany w bok. Pod spodem podpis „Streaming data from JustWatch", którego wymaga regulamin TMDB.
4. „More Like This": polecane filmy jako `MovieCardView` w wierszu przewijanym w bok. Kliknięcie otwiera szczegóły tego filmu, jak `MovieGridItem` z lab 1.

Wiersze przewijane w bok budują Państwo jak `GenreRow` z lab 1. Każdą sekcję proszę wydzielić do osobnej struktury albo właściwości, jak w `MovieDetailView`.

Testy

`MyNextMovieTests/MovieDetailsTests.swift`. JSON szczegółów The Matrix proszę wpisać w pliku testów. Powinien mieć: aktora bez zdjęcia, reżysera wymienionego dwa razy, nieoficjalny zwiastun przed oficjalnym, w Polsce wydanie z pustą `certification` przed wydaniem z „16” oraz w Polsce serwis w abonamencie i serwis do wypożyczenia. Drugi, krótki JSON to film zapowiedziany: `"runtime": null` i puste listy.
Struktura `MovieDetailsDecodingTests`:

Test	**Sprawdza**
`decodesTaglineAndRuntime`	`id`, `tagline`, `runtime` po dekodowaniu
`decodesCastWithAndWithoutPhoto`	pierwszy aktor ma `profilePath`, drugi ma `nil`
`decodesWatchProvidersUnderSlashKey`	serwisy w PL: `flatrate` i `rent` są, `buy` to `nil`
`decodesCountryCodeOfReleaseDates`	`countryCode` to „PL”
`decodesRecommendationsAsMovies`	polecane filmy to zwykłe `Movie`
`decodesAnnouncedMovieWithoutRuntime`	`"runtime": null` daje `nil`, pusty `formattedRuntime` i brak serwisów

Struktura `MovieDetailsTests`:

Test	**Sprawdza**
`runtimeInHoursAndMinutes`	136 → „2h 16m”, 120 → „2h 0m”
`runtimeUnderAnHourHasOnlyMinutes`	45 → „45m”
`runtimeIsEmptyWhenUnknown`	`nil` i 0 dają `""`
`directorsAreListedOnceEach`	„Lana Wachowski, Lilly Wachowski”
`directorsAreEmptyWithoutDirector`	bez reżysera `""`
`certificationSkipsEmptyOnes`	„16” dla PL, „R” dla US
`certificationIsEmptyForOtherCountry`	kraj bez danych daje `""`
`officialTrailerWins`	klucz oficjalnego zwiastuna
`unofficialTrailerWhenNoOfficial`	bez oficjalnego klucz nieoficjalnego
`trailerOnOtherSiteIsSkipped`	zwiastun spoza YouTube się nie liczy
`trailerURLPointsToYouTube`	adres `https://www.youtube.com/watch?v=` plus klucz
`factsLineAddsRuntimeAndCertification`	„1999 · ★ 8.2 · 2h 16m · 16”
`factsLineBeforeDetailsLoad`	z `emptyMovieDetails` samo „1999 · ★ 8.2”
`profileURLIsNilWithoutPhoto`	adres zdjęcia albo `nil`

Krótkie szczegóły do testów funkcji robi pomocnicza funkcja w pliku testów, np. `makeMovieDetails(runtime:videos:)`, tak jak `makeMovie` w `MovieTests.swift`.

`MyNextMovieTests/MovieDetailViewModelTests.swift`,
jak `MovieListViewModelTests`:

Test	Sprawdza
`startsIdleWithEmptyDetails`	stan `.idle`, pusta obsada
`loadsDetailsOfItsMovie`	closure dostaje id filmu z `init`, stan `.loaded`
`showsErrorWhenLoadingFails`	`NoConnection()` daje stan `.failed` i „No connection”
`loadIfNeededLoadsOnlyOnce`	dwa razy `loadIfNeeded`, jedno wywołanie closure

Proszę sprawdzić w aplikacji:

1. Szczegóły filmu pokazują hasło, czas trwania z kategorią wiekową i reżysera.
2. „Watch Trailer” otwiera zwiastun.
3. Obsada przewija się w bok.
4. „Where to Watch” pokazuje serwisy, gdy film jest w Polsce w streamingu.
5. „More Like This” otwiera szczegóły kolejnego filmu. Strzałka wstecz wraca do poprzedniego.
6. Szczegóły działają też z ekranu Search.

Zadanie 3: bez przykładowych danych

Aplikacja ma już prawdziwe dane, więc `Services/SampleMovies.swift` znika. Przed usunięciem proszę przenieść to, co z niego korzystało.

Testy: `MyNextMovieTests/TestMovies.swift`

- `testMovies`: filmy wpisane w testach, tak jak dotąd `sampleMovies`. Te same pięć filmów i jeszcze dwa: Knives Out (komedia, kryminał, zagadka) i Titanic (dramat, romans). Proszę wpisać im prawdziwe `posterPath` z TMDB.
- `testMovieDetails`: szczegóły The Matrix, skrócone. Dla `MovieDetailViewModelTests`.

Plik ma `@testable import MyNextMovie`, a obie stałe są `@MainActor`, jak testy.

W testach z lab 1 i lab 2 proszę zamienić `sampleMovies` na `testMovies`. Po zamianie wszystkie dotychczasowe testy dalej przechodzą.

Podglądy: `Views/Components/PopularMoviesPreview.swift`

Podglądy pytają TMDB, tak jak aplikacja. Potrzebują więc klucza w `Secrets.xcconfig`.

- Podgląd całego ekranu dostaje funkcje TMDB, np. `MovieListView(viewModel: MovieListViewModel(loadMovies: fetchPopularMovies), loadDetails: fetchMovieDetails)`.
- Podgląd jednego filmu potrzebuje filmu, który trzeba najpierw pobrać. Robi to widok `PopularMoviesPreview`: w `.task` woła `fetchPopularMovies()`, w trakcie pokazuje `ProgressView`, po błędzie jego tekst, a po załadowaniu treść z parametru `content: ([Movie]) -> Content`:

```
#Preview {
    PopularMoviesPreview { movies in
        MovieCardView(movie: movies[0])
    }
}
```

`PopularMoviesPreview` jest generyczny: `struct PopularMoviesPreview<Content: View>: View`, a `content` ma atrybut `@ViewBuilder`.

Do zmiany są podglądy w `PosterView`, `RatingBadge`, `MovieCardView`, `MovieDetailView`, `MovieListView` i `SearchView`.

Szare karty w `MovieListView`

W czasie ładowania lista pokazywała `sampleMovies` jako szare kształty. Teraz proszę zrobić je z pustych filmów: prywatna funkcja `makePlaceholderMovies()` zwraca 6 filmów z kolejnymi `id` i tytułem „Movie title". Liczba i tytuł to stałe. `.redacted(reason: .placeholder)` rysuje tekst jako szary pasek, więc liczy się tylko jego długość.

Na koniec proszę usunąć `SampleMovies.swift`. Projekt ma się zbudować bez ostrzeżeń, a wszystkie testy mają przechodzić.

Dokumentacja

- TMDB, popularne filmy: https://developer.themoviedb.org/reference/movie-popular-list
- TMDB, wyszukiwanie: https://developer.themoviedb.org/reference/search-movie
- TMDB, discover: https://developer.themoviedb.org/reference/discover-movie
- TMDB, szczegóły filmu: https://developer.themoviedb.org/reference/movie-details
- TMDB, append_to_response: https://developer.themoviedb.org/docs/append-to-response
- TMDB, serwisy streamingowe: https://developer.themoviedb.org/reference/movie-watch-providers
- `URLComponents`: https://developer.apple.com/documentation/foundation/urlcomponents
- `URLSession`: https://developer.apple.com/documentation/foundation/urlsession
- `JSONDecoder`: https://developer.apple.com/documentation/foundation/jsondecoder
- Kodowanie i dekodowanie własnych typów: https://developer.apple.com/documentation/foundation/encoding-and-decoding-custom-types
- `AsyncImage`: https://developer.apple.com/documentation/swiftui/asyncimage
- `Link`: https://developer.apple.com/documentation/swiftui/link
- `redacted(reason:)`: https://developer.apple.com/documentation/swiftui/view/redacted(reason:)
- Swift Testing, oczekiwane błędy: https://developer.apple.com/documentation/testing/testing-for-errors-in-swift-code